

Conceptual Development Document

Autonomous ER Edge AI Agent for Real-Time Clinical Notes & Alerts

The project idea I have chosen is ER note-taking. The main issue I want to address is how doctors are constantly overloaded with treating patients, monitoring vitals, documentation, etc. That documentation part takes up a lot of time that can be spent helping patients.

The main idea is to design an AI system that can basically act like a smart secretary in the ER. It would collect patient data in real time, take input from doctors (voice or typed), and then automatically generate notes. While doing all this, it would also be able to determine whether the patient is high risk or not.

System Architecture

First come the data sources, which include things like sensors (heart rate, blood pressure, oxygen levels) and also doctor input like voice recordings or notes.

Next, all of this data goes to an edge device. The reason for the data going to an edge device and not the cloud is speed and privacy. Instead of sending everything to the cloud, the system processes data locally so it's faster and more secure.

Next is the AI layer which does the following:

- Converts speech to text
- Extracts symptoms and important information
- Generates ER notes
- Classifies patient risk level
- Compares with similar past cases

Finally, there's the actual agent, which uses AI as well as simple rules. For example:

- If oxygen is too low, it triggers an alert immediately
- If symptoms suggest something serious, it marks the patient as high risk
- If everything looks normal, it just generates a routine note

All of this is shown on a dashboard for doctors, so they can quickly see what's going on.

Data Flow

The flow starts with sensors as well as doctor input (data collection.) Next, the data collected from those sources is passed on to an edge device. Afterwards comes the AI processing where it converts speech to text, generates notes, classifies risk level, etc, as mentioned above. Next is the decision agent which makes certain decisions/classifications based on the condition of the patient. Finally, all this data is displayed on a dashboard so doctors can easily view them and understand the patient's situation.

The system would use tools like MQTT for sensor data and APIs for communication, and everything would be encrypted for security.

AI Models Used

The AI layer is a combination of different types of models:

- NLP (Natural Language Processing) - to extract symptoms and info
- Generative AI - to write ER notes
- Risk classification model - to determine how serious the case is
- Retrieval system - to compare with similar cases

Thus, it's not just one model doing all the heavy lifting. Rather, multiple components work together to achieve the desired result.

Also, since real medical data is hard to access, the system would be tested using synthetic data.

Security & Ethics

This part is extremely important because we're dealing with patient data.

In terms of security, firstly, data would be encrypted and thus kept secure. Next, only authorized staff as well as devices would be able to access it (zero trust). Finally, everything would be logged and documented.

In terms of ethics, there are also a few clarifications that need to be made. Firstly, the AI is not making final decisions. A machine making final decisions over people's lives would be extremely controversial and risky. It is all reviewed by doctors. Furthermore, the system must explain alerts and the reasoning behind. Finally, the AI system should be tested on a variety of different patients to ensure that no bias exists.

Implementation Plan

If I were to actually build this, my workflow would look like the following:

I would initially create synthetic patient data (perhaps through an LLM like ChatGPT). Next, I would send the data through a pipeline (either API or MQTT) to an edge device where I would have the data processed. The AI system would then extract symptoms and determine the risk level. Next, it would generate notes after which, it would display everything on a dashboard.

Testing Strategy

To make sure it works, I'd first test the system's performance to see how fast it responds. Then I would check its accuracy to see how correct/appropriate its outputs and decisions are. Next, I would check how the system performs under pressure when it needs to handle multiple patients. Finally, I would test its level of protection under attacks for security. In all cases, I would test to see how fair and impartial the system performs.

Challenges

There are definitely some trade-offs:

- Bigger models are more accurate but slower
- Smaller models are faster but less accurate
- Privacy vs accessibility
- Trust (doctors might not fully trust AI at first)

So the system has to balance all of that.

Conclusion

Overall, this project shows how AI could actually make a real difference in emergency rooms. Instead of replacing doctors, it helps them by reducing the time spent on documentation and by catching high-risk cases faster.

Using edge computing also makes it faster and more secure, which is really important in healthcare.